

- **Student 1 Name:** Rana Hegazy Sedki
- **Student 2 Name:** Asmaa Ali Abdelwahab

Problem 1 — Hello World MPI Program

Problem Title: Hello World MPI Program

Lecture Number: MPI (Example 4.1)

1.1 Problem Description

The objective is to write the simplest possible MPI program in C. Each process prints a greeting message that includes its rank (process ID) and the total number of processes. The program exercises the four foundational MPI operations: `MPI_Init`, `MPI_Comm_size`, `MPI_Comm_rank`, and `MPI_Finalize`.

1.2 Parallelization Approach

Every process executes the same code (SPMD — Single Program Multiple Data). `MPI_Init` creates the parallel execution environment; `MPI_Comm_size` queries the total process count; `MPI_Comm_rank` returns each process's unique identity; `MPI_Finalize` shuts down the environment. No inter-process communication is required because each process independently calls `printf`.

1.3 Solution Explanation

The program follows this flow:

- Include `mpi.h` to gain access to all MPI definitions.
- Call `MPI_Init` to initialize MPI, no MPI call may precede it.
- Call `MPI_Comm_size` to know how many processes were launched.
- Call `MPI_Comm_rank` to know this process's rank (0 to size-1).
- Print the greeting, each process prints one line independently.
- Call `MPI_Finalize`, no MPI call may follow it.

The ordering of printed lines is **non-deterministic** because there is no synchronization between processes.

1.4 Source Code

```
#include <stdio.h>
#include "mpi.h"

int main(int argc, char **argv) {
    int rank, size;
    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    printf("Hello world from process %d of %d processes.\n", rank, size);
    MPI_Finalize();
    return 0;
}
```

1.5 Execution Results

```
rana@DESKTOP-71HEA74:~/my_project$ mpicc -O3 -o MPIHello hello.c
rana@DESKTOP-71HEA74:~/my_project$ mpiexec -n 3 ./MPIHello
Hello world from process 0 of 3 processes.
Hello world from process 1 of 3 processes.
Hello world from process 2 of 3 processes.
```

Each of the three processes executes `printf` once, producing three lines. The output order is **non-deterministic**.

1.6 Conclusion

The Hello World MPI program confirms that the MPI environment is correctly installed and configured. It demonstrates the four essential MPI operations and the SPMD programming model. Processes run independently with no communication overhead, making this the minimal baseline for all future MPI programs.

Problem 2 — Ping-Pong Message Transfer

Problem Title: Ping-Pong Message Transfer

Lecture Number: MPI (Example 4.2)

2.1 Problem Description

This problem investigates MPI point-to-point messaging between exactly two processes (rank 0 and rank 1). The program sends messages of doubling size — `msg_size` initialises at 64 and doubles each iteration (first send: 128 bytes), growing up to 16,777,216 bytes (~16 MB) back and forth in a **ping-pong** pattern using **`MPI_Send`** and **`MPI_Recv`**.

The primary objectives are:

- Verify that `MPI_Send` and `MPI_Recv` work correctly for all message sizes.
- Understand how the order of `Send/Recv` calls affects program correctness.
- Demonstrate how incorrect ordering leads to a **deadlock** condition.
- Fix the deadlock by reordering calls so the two sides are always complementary.

2.2 Parallelization Approach

Two MPI processes are launched. Each process executes the same program but branches on its rank:

- **Process 0 (rank 0):** initiates the exchange by sending first, then receiving.
- **Process 1 (rank 1):** must do the complementary action — receive first, then send.

The message size doubles every loop iteration (`msg_size = msg_size × 2`), stressing both the small-message buffering regime and the large-message blocking regime. The loop runs while `msg_size < 10,000,000` bytes.

2.3 Solution Explanation

2.3.1 Why the Bug Causes a Deadlock

In the **buggy version**, Process 1 calls **`MPI_Send`** before **`MPI_Recv`**. This mirrors Process 0, which also calls `MPI_Send` first. Both processes therefore try to send at the same time:

Timeline of the deadlock:

- Process 0 calls `MPI_Send` → waits for Process 1 to post a matching `MPI_Recv`.
- Process 1 calls `MPI_Send` → waits for Process 0 to post a matching `MPI_Recv`.

- Neither side ever reaches `MPI_Recv` — both are stuck in `MPI_Send` forever.

Why small messages seem to work: MPI internally buffers short messages (the exact threshold is implementation-specific, typically around 65,536 bytes or more). This lets `MPI_Send` return early without a matching receive, masking the bug. Once messages exceed the buffer capacity the send blocks permanently — deadlock. Note: on some systems with large buffers, all messages may complete without hanging, as observed in the execution results below.

2.3.2 The Fix — Complementary Ordering

The solution is simple: in Process 1, **`MPI_Recv` must come before `MPI_Send`**. This ensures the two sides are always complementary and can never block both sides simultaneously.

With this ordering, when Process 0 calls `MPI_Send`, Process 1 is already in `MPI_Recv` waiting for it — so the transfer completes immediately. Then Process 1 calls `MPI_Send` and Process 0 calls `MPI_Recv`, completing the return trip. All message sizes up to 16 MB transfer correctly.

2.4 Source Code

2.4.1 Buggy Version (Deadlock on Large Messages)

```
int main(int argc, char *argv[]) {
    int numprocs, rank, tag = 100, msg_size = 64;
    char *buf;
    MPI_Status status;
    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &numprocs);
    if (numprocs != 2) {
        printf("The number of processes must be two!\n");
        MPI_Finalize(); return 0;
    }
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    while (msg_size < 10000000) {
        msg_size = msg_size * 2;
        buf = (char *)malloc(msg_size * sizeof(char));
        if (rank == 0) {
            MPI_Send(buf, msg_size, MPI_CHAR, 1, tag, MPI_COMM_WORLD);
            printf("Message of length %d to process %d\n", msg_size, rank+1);
            fflush(stdout);
            MPI_Recv(buf, msg_size, MPI_CHAR, 1, tag, MPI_COMM_WORLD, &status);
        }
        if (rank == 1) {
            //MPI_Recv(buf, msg_size, MPI_CHAR, rank-1, tag, MPI_COMM_WORLD, &status);
            MPI_Send(buf, msg_size, MPI_CHAR, rank-1, tag, MPI_COMM_WORLD);
            printf("Message of length %d to process %d\n", msg_size, rank-1);
            fflush(stdout);
            MPI_Recv(buf, msg_size, MPI_CHAR, rank-1, tag, MPI_COMM_WORLD, &status);
        }
        free(buf);
    }
    MPI_Finalize();
}
```

2.4.2 Fixed Version

```
int main(int argc, char *argv[]) {
    int numprocs, rank, tag = 100, msg_size = 64;
    char *buf;
    MPI_Status status;
    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &numprocs);
    if (numprocs != 2) {
        printf("The number of processes must be two!\n");
        MPI_Finalize(); return 0;
    }
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    while (msg_size < 10000000) {
        msg_size = msg_size * 2;
        buf = (char *)malloc(msg_size * sizeof(char));
        if (rank == 0) {
            MPI_Send(buf, msg_size, MPI_CHAR, 1, tag, MPI_COMM_WORLD);
            printf("Message of length %d to process %d\n", msg_size, rank+1);
            fflush(stdout);
            MPI_Recv(buf, msg_size, MPI_CHAR, 1, tag, MPI_COMM_WORLD, &status);
        }
        if (rank == 1) {
            MPI_Recv(buf, msg_size, MPI_CHAR, rank-1, tag, MPI_COMM_WORLD, &status);
            MPI_Send(buf, msg_size, MPI_CHAR, rank-1, tag, MPI_COMM_WORLD);
            printf("Message of length %d to process %d\n", msg_size, rank-1);
            fflush(stdout);
            //MPI_Recv(buf, msg_size, MPI_CHAR, rank-1, tag, MPI_COMM_WORLD, &status);
        }
        free(buf);
    }
    MPI_Finalize();
}
```

2.5 Execution Results

2.5.1 Buggy Version Output — Deadlock

The program was run with the buggy ordering (Process 1 sends before receiving).

```
rana@DESKTOP-71HEA74:~/my_project$ nano MPImessage.c
rana@DESKTOP-71HEA74:~/my_project$ mpicc -O3 -o MPImessage MPImessage.c
rana@DESKTOP-71HEA74:~/my_project$ mpiexec -n 2 ./MPImessage
Message of length 128 to process 1
Message of length 256 to process 1
Message of length 512 to process 1
Message of length 1024 to process 1
Message of length 2048 to process 1
Message of length 4096 to process 1
Message of length 128 to process 0
Message of length 256 to process 0
Message of length 512 to process 0
Message of length 1024 to process 0
Message of length 2048 to process 0
Message of length 4096 to process 0
```

Observations:

- The program **deadlocked and hung** beyond 4096 bytes on this system (buffer threshold is implementation-specific) — it never completed
- Output between rank 0 and rank 1 is **interleaved and non-deterministic**

2.5.2 Fixed Version Output — All Sizes Transfer

After correcting the order (Process 1 calls MPI_Recv before MPI_Send), the program runs to completion for all message sizes up to 16 MB:

```
rana@DESKTOP-71HEA74:~/my_project$ mpiexec -n 2 ./MPImessage
Message of length 128 to process 1
Message of length 256 to process 1
Message of length 512 to process 1
Message of length 1024 to process 1
Message of length 2048 to process 1
Message of length 4096 to process 1
Message of length 128 to process 0
Message of length 256 to process 0
Message of length 512 to process 0
Message of length 1024 to process 0
Message of length 2048 to process 0
Message of length 4096 to process 0
Message of length 8192 to process 1
Message of length 16384 to process 1
Message of length 8192 to process 0
Message of length 16384 to process 0
Message of length 32768 to process 1
Message of length 32768 to process 0
Message of length 65536 to process 1
Message of length 65536 to process 0
Message of length 131072 to process 1
Message of length 131072 to process 0
Message of length 262144 to process 1
Message of length 262144 to process 0
Message of length 524288 to process 1
Message of length 524288 to process 0
Message of length 1048576 to process 1
Message of length 1048576 to process 0
Message of length 2097152 to process 1
Message of length 2097152 to process 0
Message of length 4194304 to process 1
Message of length 4194304 to process 0
Message of length 8388608 to process 1
Message of length 8388608 to process 0
Message of length 16777216 to process 1
Message of length 16777216 to process 0
rana@DESKTOP-71HEA74:~/my_project$
```

Observations:

- All sizes from 128 bytes up to 16,777,216 bytes (16 MB) complete successfully.

- Unlike the buggy run where rank 0 printed all messages before rank 1, the fixed output shows **alternating pairs** (both ranks print each size together), demonstrating that the complementary Send/Recv ordering now synchronizes the two processes at every message boundary.

2.6 Conclusion

This experiment clearly demonstrated the **critical importance of send/receive ordering** in blocking MPI communication:

- **Root cause of deadlock:** Both processes called MPI_Send simultaneously before either called MPI_Recv. Since blocking MPI_Send does not return until a matching receive is posted, both processes blocked each other permanently.
- **Why small messages hid the bug:** MPI's internal buffering absorbed small messages (up to ~4,096 bytes on this system) making the sends return early, which is why those sizes completed. Once messages exceeded the buffer, the sends blocked permanently. The deadlock threshold is implementation-specific (the textbook reports 65,536 bytes on their system) and must never be relied upon.
- **The fix:** Ensure the two sides are always complementary. Process 0 follows Send→Recv; Process 1 must follow Recv→Send. With this ordering, one side is always ready to receive when the other side sends — deadlock is impossible.
- **Result:** After the fix, all message sizes from 128 bytes to 16,777,216 bytes transferred correctly and the program exited cleanly.

Alternative solutions that also avoid deadlock include: using **MPI_Sendrecv** (combines send and receive atomically).

Problem 3 — Communication Bandwidth Measurement

Problem Title: Measuring Communication Channel Bandwidth

Lecture Number: MPI (Example 4.3)

3.1 Problem Description

Design an MPI program that measures the communication bandwidth between two processes by sending messages of doubling length (from 1 double up to ~100 million doubles). For each length, the send/receive pair is repeated many times (nloop iterations) and the average time is used to compute bandwidth in Megabits per second.

3.2 Parallelization Approach

Two processes cooperate. Process 0 (sender) starts the wall clock, sends the message nloop times, stops the clock, and computes bandwidth. Process 1 (receiver) simply receives nloop times. The number of loop repetitions decreases as message size grows so that total test time stays reasonable.

3.3 Solution Explanation

Bandwidth = (message bytes × 8 bits/byte) / (average transfer time in seconds), expressed in Mb/s. Start-up latency dominates for short messages; raw memory bandwidth dominates for long messages. The program reveals two bandwidth regimes:

- Short messages (< 512 doubles): bandwidth grows as the fixed start-up cost t_s is amortized over more data.
- Long messages (> 500 K doubles): bandwidth stabilizes at the shared-memory or network limit.

3.4 Source Code

```
#include <stdio.h>
#include <stdlib.h>
#include "mpi.h"
#define NUMBER_OF_TESTS 10

int main(int argc, char *argv[]) {
    double *buf; int rank, numprocs, n; double t1, t2;
    int j, k, nloop; MPI_Status status;
    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &numprocs);
    if (numprocs != 2) { printf("Need 2 processes\n"); return 0; }
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    if (rank == 0) printf("\tn\ttime [sec]\tRate [Mb/sec]\n");
    for (n = 1; n < 100000000; n *= 2) {
        nloop = 1000000 / n; if (nloop < 1) nloop = 1;
        buf = (double *)malloc(n * sizeof(double));
        for (k = 0; k < NUMBER_OF_TESTS; k++) {
            if (rank == 0) {
                t1 = MPI_Wtime();
                for (j = 0; j < nloop; j++)
                    MPI_Send(buf, n, MPI_DOUBLE, 1, k, MPI_COMM_WORLD);
                t2 = (MPI_Wtime() - t1) / nloop;
            } else {
                for (j = 0; j < nloop; j++)
                    MPI_Recv(buf, n, MPI_DOUBLE, 0, k, MPI_COMM_WORLD, &status);
            }
        }
        if (rank == 0) {
            double bw = n * sizeof(double) * 1.0e-6 * 8 / t2;
            printf("\t%10d\t %10.8f\t%8.2f\n", n, t2, bw);
        }
        free(buf);
    }
    MPI_Finalize(); return 0;
}
```


3.5 Execution Results

Running on two cores of the same computer (shared memory communication):

```
rana@DESKTOP-71HEA74:~/my_project$ mpicc -O3 -o Bandwidth Bandwidth.c
rana@DESKTOP-71HEA74:~/my_project$ mpiexec -n 2 ./Bandwidth
```

n	time [sec]	Rate [Mb/sec]
1	0.00000013	501.06
2	0.00000023	560.22
4	0.00000013	1949.44
8	0.00000027	1878.84
16	0.00000016	6593.25
32	0.00000017	12014.72
64	0.00000019	21280.13
128	0.00000019	42892.70
256	0.00000023	71264.93
512	0.00000032	102164.60
1024	0.00000241	27210.25
2048	0.00000283	46376.98
4096	0.00000395	66400.02
8192	0.00000554	94592.04
16384	0.00000960	109245.32
32768	0.00001557	134691.84
65536	0.00002928	143248.09
131072	0.00007906	106108.16
262144	0.00012087	138807.63
524288	0.00026020	128956.31
1048576	0.00057090	117549.25
2097152	0.00120410	111467.26
4194304	0.00320740	83692.54
8388608	0.00532150	100887.14
16777216	0.00945690	113540.57
33554432	0.01976010	108677.77
67108864	0.04183760	102658.07

Bandwidth peaks at 65,536 doubles (143,248 Mb/s $\approx$ 143 Gb/s). Beyond that it drops and then stabilises in the range of approximately 83,000–114,000 Mb/s for large messages (>4 million doubles), limited by the shared memory bandwidth. All results are in Mb/s as printed by the program.

3.6 Conclusion

MPI_Wtime provides portable, high-resolution wall-clock timing. The bandwidth curve confirms that short messages are dominated by start-up latency ($\sim 0.13 \mu\text{s}$, from the $n=1$ measurement) while long messages reach the underlying hardware limit. This measurement is essential for choosing the right message granularity in real parallel applications.